

# Sistem Programlama Finali

## Slaytlardan Ders Notu + Çıkış Soru Noktaları + Çözümler

### Bu not nasıl kullanılacak?

- 1) Her başlık önce slayttaki bilgiyi özetler.
- 2) Hemen altında o konudan çıkış sorular belirtilir.
- 3) Her çıkış soru için kısa çözüm mantığı verilir.
- 4) Son sayfalarda ezber tabloları ve cevaplı mini test vardır.

Not: Çıkış soru fotoğraflarında bazı şıklar flu olduğu için notta asıl kavram ve çözüm mantığı öne çıkarılmıştır.

### Sınavda En Çok Gelen 12 Konu

| Öncelik | Konu                                                | Neden önemli?                                                                   |
|---------|-----------------------------------------------------|---------------------------------------------------------------------------------|
| 1       | Shell redirection: >, >>, <, 2>, stdout/stderr      | Çıkışlarda cat/echo çıktısı, hata nereye gider, dosya içeriği ne olur sorulmuş. |
| 2       | open/read/write/close/lseek ve file descriptor      | O_CREAT/O_TRUNC/O_WRONLY, fd=3, EBADF, read null-terminate tuzakları var.       |
| 3       | chmod, umask, octal permission                      | umask 077, chmod 600/0666, atoi("0744") tuzağı çıkış.                           |
| 4       | Hard link, soft link, inode, unlink/remove          | ln, ln -s, stat.st_nlink, soft link ile hard link farkı sorulmuş.               |
| 5       | stat/lstat, opendir/readdir/closedir, struct dirent | d_name, st_size, dizi gezme, . ve .. atlama soruları var.                       |
| 6       | C pointer, string, malloc/stack/global              | strchr, strdup, pointer->int veri kaybı, segfault, malloc adres sırası çıkış.   |
| 7       | Struct/union ve padding                             | struct kaç byte, union kaç byte, hizalama mantığı sorulmuş.                     |
| 8       | Signal ve process fonksiyonları                     | Ctrl+C, signal(SIGINT,...), getpid/getuid/getgid/getppid sorulmuş.              |
| 9       | Buffering ve performans                             | getchar/putchar, read/write, fread/fwrite performans sıralaması sorulmuş.       |
| 10      | Pseudo assembler: r0, sp, fp, pc                    | Fonksiyon dönüşü, parametre ve local adresleri çıkış.                           |
| 11      | Pseudo assembler: C kodu assembly eşleştirme        | main(), a(int *p), p[3], if/else, push/jsr/pop/st kalıpları sorulmuş.           |
| 12      | Makefile/derleme-bağlama                            | Sınavda az ama final slaytlarında var; compile/link ayrımı bilinmeli.           |

# 1. UNIX, Dosya Sistemi ve Temel Komut Mantığı

*Slayt kaynağı: Hafta8: dosya sistemi, adlar, yollar, program/proses; Hafta10: cat, buffering, files/links/inodes.*

## Slayttan Ders Notu

| Kavram        | Bilmen gereken                                                 |
|---------------|----------------------------------------------------------------|
| Dosya sistemi | Dizinlerin hiyerarşik düzenidir. Unix kök dizini / ile başlar. |
| Dosya adı     | Bir dosyanın bir dizinde görüldüğü isimdir.                    |
| Pathname      | Slash / ile ayrılan dosya ve dizin adları dizisidir.           |
| Mutlak yol    | / ile başlar. Örnek: /home/sau/test.txt                        |
| Görelî yol    | / ile başlamaz. Mevcut çalışma dizinine göre yorumlanır.       |
| pwd           | Çalışma dizininin tam yolunu verir.                            |
| cd            | Çalışma dizinini değiştirir.                                   |
| ls -a         | . ve .. dahil bütün dosya adlarını listeler.                   |
| Program       | Yürütülebilen veya derlenmesi gereken dosya.                   |
| Proses        | Programın çalışmakta olan örneği.                              |

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: /etc dizininde ne tür dosyalar vardır?

Çözüm: /etc genelde sistem yapılandırma dosyalarını tutar. Eski çıkmışta /etc için "system configuration files" mantığı sorulmuştu.

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: grep -r "error" \*.log komutu ne yapar?

Çözüm: grep metin arar. -r recursive aramadır. \*.log kalıbındaki log dosyalarında error geçen satırları arar.

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: tail -20 hasta.c komutu ne yapar?

Çözüm: hasta.c dosyasının son 20 satırını gösterir.

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: cp -r sistem yedeksistem komutu ne yapar?

Çözüm: -r recursive kopyadır. sistem dizinini içeriğiyle birlikte yedeksistem hedefine kopyalar.

### Sınav taktiği

Komut sorularında her satırdan sonra dosya/dizin tablosu tut: dosya adı, içerik, izin, link durumu, hata çıktısı.

## 2. Shell Redirection: stdout, stderr, >, >>, <, 2>

*Slayt kaynağı: Hafta11 slaytlarında yönlendirme ilkelleri: > standart çıktıyı dosyaya yazar, >> ekler, < dosyadan standart girdi alır.*

## Slayttan Ders Notu

| Yazım              | Anlamı                                          | Tuzak                                                 |
|--------------------|-------------------------------------------------|-------------------------------------------------------|
| komut > f          | stdout dosyaya yazılır, dosya varsa sıfırlanır. | stderr ekrana gitmeye devam eder.                     |
| komut >> f         | stdout dosyanın sonuna eklenir.                 | Eski içerik silinmez.                                 |
| komut < f          | stdin f dosyasından alınır.                     | Program klavyeden değil dosyadan okur.                |
| komut 2> err       | stderr err dosyasına gider.                     | stdout ayrı kalır.                                    |
| komut > out 2> err | stdout out, stderr err dosyasına gider.         | İki akış ayrılır.                                     |
| komut > f1 > f2    | Son stdout yönlendirmesi geçerlidir.            | f1 oluşturulabilir/sıfırlanabilir, çıktı f2'ye gider. |

0 = stdin  
1 = stdout  
2 = stderr

## ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: cat f1 f3 > f5; f1 var ama f3 yoksa ne olur?

Çözüm: f1 içeriği stdout olduğu için f5 dosyasına gider. f3 bulunamadı hatası stderr olduğu için ekrana yazılır. Çünkü > sadece descriptor 1'i yönlendirir, descriptor 2'ye dokunmaz.

## ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: cat f1 > f2 > f5 gibi çoklu yönlendirme sorusu

Çözüm: Kabuk soldan sağa dosyaları açar ama stdout en son nereye bağlandıysa çıktı oraya gider. Bu yüzden gerçek çıktı son dosyaya gider; önceki dosya boş oluşturulabilir/sıfırlanabilir.

## ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: Program < dosya1 > dosya2 sorusunda yanlış ifade hangisi?

Çözüm: stdin dosya1'den okunur, stdout dosya2'ye yazılır. Program ekran çıktısını doğrudan terminale değil dosya2'ye gönderir. Hata stderr ise terminalde kalabilir.

## 3. System Call, Standard I/O ve File Descriptor

Slayt kaynağı: Hafta8: open/read/write/close; Hafta10: system calls integer fd ile, standard I/O FILE\* ile çalışır; buffering çıkarımları.

## Slayttan Ders Notu

| Fonksiyon | Tür         | Ne yapar?                                                     |
|-----------|-------------|---------------------------------------------------------------|
| open()    | System call | Dosya açar, başarılıysa integer file descriptor döndürür.     |
| read()    | System call | fd'den byte okur. C string sonuna otomatik \0 koymaz.         |
| write()   | System call | fd'ye byte yazar.                                             |
| close()   | System call | fd ile iş bittiğini OS'ye bildirir; fd tekrar kullanılabilir. |
| lseek()   | System call | Dosya offset/read-write pointer konumunu değiştirir.          |

| Fonksiyon                   | Tür         | Ne yapar?                                                            |
|-----------------------------|-------------|----------------------------------------------------------------------|
| stat/lstat()                | System call | Dosya metadata/inode bilgilerini alır.                               |
| printf/fprintf/fopen/fclose | C library   | FILE* üstünden buffered G/Ç yapar. Doğrudan sistem çağrısı değildir. |
| opendir/readdir/closedir    | C library   | Dizin okumak için library çağrılarındır; altta system call kullanır. |

## open bayrakları

| Bayrak   | Anlam                                                 |
|----------|-------------------------------------------------------|
| O_RDONLY | Sadece okumak için açar.                              |
| O_WRONLY | Sadece yazmak için açar.                              |
| O_RDWR   | Okuma + yazma için açar.                              |
| O_CREAT  | Dosya yoksa oluşturur. Üçüncü parametre mode gerekir. |
| O_TRUNC  | Dosya varsa içeriğini sıfır bayta indirir.            |
| O_APPEND | Yazmaları dosyanın sonuna ekler.                      |

```
int fd = open("d1.txt", O_WRONLY | O_CREAT | O_TRUNC, 0600);
```

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: open(..., O\_WRONLY | O\_CREAT | O\_TRUNC, 0600) hangi işi yapar?

Çözüm: Dosyayı sadece yazma modunda açar. Dosya yoksa oluşturur. Varsa içeriğini siler. İzin mode değeri 0600 ise sahibi okur/yazar, grup ve diğerleri erişemez.

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: write(3, s, strlen(s)) çağrısı ve fd=3 açık değilse hangi hata?

Çözüm: Program fd=3'ü open ile açmamışsa 3 geçerli fd değildir. write -1 döner ve perror çoğunlukla "Bad file descriptor/Bad file number" mesajı basar.

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: Aşağıdakilerden hangisi sistem çağrısı değildir? fprintf/printf/malloc seçenekleri

Çözüm: fprintf ve printf C library fonksiyonudur; malloc da C library/heap ayırma fonksiyonudur. open, read, write, close, fork, link, stat, lseek system call olarak kabul edilir.

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: Büyük dosya stdin'den okunup stdout'a yazılacaksa performans sıralaması

Çözüm: Tek byte raw read/write çok sistem çağrısı üretirse en kötüdür. C stdio buffering kullanan getchar/putchar ve özellikle fread/fwrite daha iyidir. Büyük bloklarda fread/fwrite veya read/write buffer ile hızlıdır; buffersız tek byte read/write kötü olur.

## 4. Dosya İzinleri: chmod, umask, Octal

Slayt kaynağı: Hafta12: umask ve chmod örnekleri; açık fd varken chmod'un açık fd'ye etkisi olmadığı örneği.

## Slayttan Ders Notu

| Sembol | Değer | Anlam                            |
|--------|-------|----------------------------------|
| r      | 4     | Okuma                            |
| w      | 2     | Yazma                            |
| x      | 1     | Çalıştırma / dizin için traverse |
| 7      | 4+2+1 | rwx                              |
| 6      | 4+2   | rw-                              |
| 5      | 4+1   | r-x                              |
| 4      | 4     | r--                              |

Gerçek izin = istenen\_mode & ~umask  
 Dosya default: 0666  
 Dizin default: 0777

| Örnek                      | Hesap                           | Sonuç            |
|----------------------------|---------------------------------|------------------|
| umask 022, dosya mode 0666 | 0666 & ~022                     | 0644 = rw-r--r-- |
| umask 077, dosya mode 0666 | 0666 & ~077                     | 0600 = rw-----   |
| umask 077, dizin mode 0777 | 0777 & ~077                     | 0700 = rwx-----  |
| chmod 600 f                | owner=rw, group=---, other=---  | -rw-----         |
| chmod 755 d                | owner=rwx, group=r-x, other=r-x | rwxr-xr-x        |

**ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM**

Sorulan çıkmış nokta: umask 077 iken kullanıcı veri.txt oluşturursa users grubu ne yapabilir?

Çözüm: Dosya default 0666 kabul edilir. 0666 & ~077 = 0600. Grup izinleri 0 olur. users grubundaki kullanıcıların dosyaya erişimi yoktur.

**ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM**

Sorulan çıkmış nokta: umask 333, echo sistem > d2.txt sonrası izin ne olur?

Çözüm: Dosya default 666'dır. 666 & ~333 = 444. Sonuç -r--r--r-- olur. Fotoğraftaki d2.txt sorusu bu mantıktır.

**ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM**

Sorulan çıkmış nokta: chmod() ile izin ----- yapılsa bile daha önce open edilmiş fd yazabilir mi?

Çözüm: Evet. fd geçerli olarak açıldıysa chmod açık dosya tanımlayıcısını iptal etmez. Aynı proses fd ile yazmaya devam edebilir; ama yeni cat/open izin hatası alır.

**ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM**

Sorulan çıkmış nokta: char mode[]="0744"; i=atoi(mode); chmod(buf,i) neden hatalı?

Çözüm: atoi("0744") decimal 744 üretir; permission için octal 0744 gerekir. Doğrusu sabit 0744 kullanmak veya strtol(mode, NULL, 8) ile 8 tabanında çevirmektir.

**5. Dosyalar, Inode, Hard Link, Soft Link, unlink/remove**

Slayt kaynağı: Hafta10: dosya diskte izin girişi + inode + data blokları olarak anlatılır. Hafta12: remove/read açık fd örneği.

Son gece tekrar notu - çıkmış soru odaklı

## Slayttan Ders Notu

| Kavram        | Özet                                                                                          |
|---------------|-----------------------------------------------------------------------------------------------|
| Dizin girişi  | Dosya adını inode numarası ile ilişkilendirir.                                                |
| Inode         | Boyut, sahip, izin, link sayısı, zaman bilgileri ve data blok adresleri gibi metadata içerir. |
| Data blok     | Dosyanın gerçek byte içerikleri burada tutulur.                                               |
| Hard link     | Aynı inode'a ikinci dosya adı verir. Link sayısı artar.                                       |
| Soft link     | Hedef dosyanın yolunu tutan ayrı bir dosyadır. Inode'u ayrıdır.                               |
| unlink/remove | Dizin girişini kaldırır, link sayısını azaltır. Son fd kapanana kadar veri silinmeyebilir.    |

```
ln f1 f2      # hard link
ln -s f1 s1   # symbolic/soft link
unlink("f1") # rm f1 mantığı, link count azaltır
```

## ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: d1 dosyasına s1 isimli sembolik link nasıl oluşturulur?  
Çözüm: Doğru komut: ln -s d1 s1. Hard link için ln d1 s1 olurdu.

## ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: Hard link ve soft link için yanlış/doğru ifadeler  
Çözüm: Hard link aynı inode numarasına sahiptir. Soft link hedef yolunu tutar, inode'u farklıdır. Soft link sayısı artınca hedef dosyanın disk alanı değişmez. Bir dosyayı gösteren tek hard link silinirse ve açık fd yoksa dosya silinir.

## ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: stat ile hard link sayısı nasıl bulunur?  
Çözüm: struct stat buf; stat("f1.txt", &buf); printf("%ld", (long)buf.st\_nlink); st\_nlink hard link sayısını verir.

## ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: Açık dosya remove() edilirse read() çalışır mı?  
Çözüm: Eğer dosya önceden açılmış ve fd geçerliyse, izin girişi silinse bile son fd kapanana kadar inode/data tutulabilir. Bu yüzden read başarılı olabilir.

## 6. stat, lstat, opendir, readdir, closedir ve Recursive Dizin Gezme

**Slayt kaynağı:** Hafta11: stat dosyanın inode bilgisini alır; opendir/readdir/closedir dizin okumada kullanılır. Hafta12: prsize recursive traversal ve . / .. tuzağı.

## Slayttan Ders Notu

| Fonksiyon / Yapı  | Ne işe yarar?                                                    |
|-------------------|------------------------------------------------------------------|
| stat(path, &buf)  | Hedef dosyanın metadata/inode bilgisini doldurur.                |
| lstat(path, &buf) | Path soft link ise hedef yerine linkin kendisini inceler.        |
| struct stat       | st_mode, st_size, st_ino, st_nlink, st_uid, st_gid, zamanlar vb. |
| S_ISDIR(m)        | mode bilgisinden dosya izin mi kontrol eder.                     |

| Fonksiyon / Yapı | Ne işe yarar?                                                             |
|------------------|---------------------------------------------------------------------------|
| S_ISLNK(m)       | mode bilgisinden sembolik link mi kontrol eder. Bunun için lstat gerekir. |
| opendir(path)    | Dizini açar, DIR* döndürür.                                               |
| readdir(DIR*)    | Sıradaki dizin girişini struct dirent* olarak verir.                      |
| closedir(DIR*)   | Dizini kapatır.                                                           |
| struct dirent    | d_name alanı dosya/dizin adını verir.                                     |

```
DIR *d = opendir(".");
struct dirent *de;
while ((de = readdir(d)) != NULL) {
    printf("%s\n", de->d_name);
}
closedir(d);
```

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkış nokta: Bir dizinin içindeki dosya ve dizin adlarına erişmek için hangi veri yapısı gerekir?

Çözüm: struct dirent gerekir. readdir() her çağrıda struct dirent\* döndürür; dosya adı de->d\_name alanındadır.

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkış nokta: Dosyalar hakkında bilgi almak için kullanılan sistem çağrısı hangisidir?

Çözüm: stat. Dosyanın boyutu, inode numarası, hard link sayısı, mod/izin bilgisi gibi metadata alınır.

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkış nokta: struct stat ile hangi bilgi elde edilemez?

Çözüm: Koruma bilgisi st\_mode, hard link sayısı st\_nlink, soft link sayısı doğrudan yoktur, dosya boyutu st\_size, inode numarası st\_ino. Soft link sayısı diye doğrudan alan yoktur.

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkış nokta: Recursive directory traversal neden . ve .. atlamalı?

Çözüm: . geçerli dizin, .. üst dizindir. Bunları tekrar gezerse sonsuz recursion veya fd tükenmesi oluşur. Bu prsize örneklerinde anlatılan kritik tuzaktır.

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkış nokta: Dizin içindeki dosya boyutunu sadece readdir ile bulabilir miyim?

Çözüm: readdir dosya adını verir; boyut için stat/lstat ile ayrıca metadata alınmalıdır. de->d\_name ad verir, st\_size stat sonucundadır.

## 7. C Stringleri, fgets/strchr/strdup/strcpy ve read() Tuzağı

Slayt kaynağı: Hafta5: string işlemleri; Hafta8: read() null sonlandırmaz; 2019-2020 çıkmışta strchr/ptr-line ve strdup soruları var.

### Slayttan Ders Notu

| Fonksiyon | Anlam / Tuzak                                             |
|-----------|-----------------------------------------------------------|
| strlen(s) | \0 karakterine kadar uzunluğu verir; \0 sonrası sayılmaz. |

| Fonksiyon            | Anlam / Tuzak                                                                        |
|----------------------|--------------------------------------------------------------------------------------|
| strcpy(dst, src)     | src stringini dst'ye kopyalar. dst yeterli alanlı olmalı.                            |
| strcat(dst, src)     | src'yi dst'nin sonuna ekler. dst'de yeterli alan yoksa taşma olur.                   |
| strchr(s, c)         | s içinde c karakterinin ilk adresini döndürür; yoksa NULL.                           |
| fgets(buf, n, stdin) | Satır okur, çoğunlukla newline karakterini de tutar.                                 |
| strdup(s)            | strlen(s)+1 kadar malloc yapar, s'yi kopyalar, yeni pointer döndürür.                |
| read(fd, buf, n)     | Byte okur; metin okuyunca buf[n] = \0 yapmazsan printf string gibi güvenli değildir. |

```
char *strdup(char *s) eşdeğeri:
char *p = malloc(strlen(s) + 1);
strcpy(p, s);
return p;
```

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: strdup(char \*s) çağrısına eşdeğer seçenek hangisi?

Çözüm: strcpy(malloc(strlen(s)+1), s). Çünkü \0 karakteri için +1 byte ayrılır ve string kopyalanır. strcmp değildir; strcmp karşılaştırır.

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: fgets ile satır okunup ptr = strchr(line, ' ') yapılırsa ptr-line ne verir?

Çözüm: İlk boşluk karakterinin indeksini verir. "Sakarya Üniversitesi" satırında ilk boşluk Sakarya'dan sonra olduğundan indeks 7'dir.

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: strlen("abc\0def") kaçtır?

Çözüm: 3. C string ilk \0 karakterinde biter.

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: read() sonrası printf("%s", buf) neden tehlikelidir?

Çözüm: read byte okur ama sonuna \0 koymaz. String gibi basmak için okunan byte sayısı n ise buf[n] = \0 yapılmalıdır.

## 8. C Bellek Bölgeleri, Pointer ve Adres Mantığı

Slayt kaynağı: Hafta2: sizeof, pointer boyutu, 32/64 bit farkı; Hafta3/4: pointer aritmetiği, struct yerleşimi, segfault.

### Slayttan Ders Notu

| Bölge           | Örnek                | Genel adres mantığı                                                              |
|-----------------|----------------------|----------------------------------------------------------------------------------|
| Code/Text       | Program komutları    | Genelde düşük/ayrı adres bölgesi.                                                |
| Global/Data/BSS | global int c;        | Statik ömürlü değişkenler; heap/stackten farklıdır.                              |
| Heap            | malloc(8)            | Dinamik bellek. malloc pointerı genelde heap bölgesindedir.                      |
| Stack           | main içindeki int b; | Local değişkenler ve fonksiyon frame bilgileri. Genelde yüksek adreslerden aşağı |



| Bölge | Örnek | Genel adres mantığı |
|-------|-------|---------------------|
|       |       | büyür.              |

| İfade                     | Anlam                                                               |
|---------------------------|---------------------------------------------------------------------|
| int *p; p+1               | sizeof(int) kadar ilerler, genelde 4 byte.                          |
| char *p; p+1              | 1 byte ilerler.                                                     |
| a[i]                      | *(a+i) ile aynıdır.                                                 |
| sizeof(int *)             | Pointer boyutu; 64 bitte genelde 8, 32 bitte 4.                     |
| Pointerı int'e cast etmek | 64 bitte pointer 8 byte, int 4 byte olduğundan veri kaybı olabilir. |

**ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM**

Sorulan çıkmış nokta: malloc adresi, local değişken adresi ve global değişken adresi sıralaması

Çözüm: Tipik 64-bit Linux'ta stack adresleri yüksek, heap daha aşağı, global/data daha aşağıdır. Bu yüzden local değişken adresleri genelde malloc adresinden büyük olabilir. Soruda mantık: a=malloc heap, bp=&b stack, cp=&c global/data.

**ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM**

Sorulan çıkmış nokta: 64 bit sistemde char \*s pointerı int i'ye atanırsa hangi satırda veri kaybı olur?

Çözüm: i = (int) s satırında veri kaybı olur. Çünkü pointer 8 byte, int 4 byte olabilir. Daha sonra tekrar char\* yapılması kaybolan üst bitleri geri getirmez.

**ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM**

Sorulan çıkmış nokta: int \*p ile p+1 kaç byte ilerler?

Çözüm: int boyutu kadar ilerler; bu derste genelde 4 byte kabul edilir. Pointer arithmetic byte değil eleman boyutuna göre dir.

## 9. Segmentasyon İhlali, Bus Error ve Hata Mantığı

*Slayt kaynağı: Hafta4: erişilemeyen belleğe erişmek segmentasyon ihlali; hizalanmamış skaler erişim bus error oluşturabilir.*

### Slayttan Ders Notu

| Hata               | Sebebi                                                                   | Çıkış bağlantısı              |
|--------------------|--------------------------------------------------------------------------|-------------------------------|
| Segmentation fault | Erişilemeyen adrese erişmek, NULL pointer dereference, geçersiz pointer. | char *ch=0; *ch=3; sorusu.    |
| Bus error          | Bazı mimarilerde hizalanmamış adresten int/double gibi skaler okumak.    | Misalignment soruları.        |
| Data loss          | Büyük türü küçük türe cast etmek, örn. pointer -> int.                   | 64-bit pointer to int çıkmış. |

```
void foo() {
    char *ch = 0;
    *ch = 3;
}
// Derlenir, çalışırken segmentation fault verir.
```

**ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM**

Sorulan çıkmış nokta: char \*ch = 0; \*ch = 3; olan program derlenir mi, çalışınca ne olur?

Çözüm: Kod sözdizimi olarak derlenebilir. Çalışırken 0/NULL adresine yazmaya çalışır ve segmentation fault verir. "Tamamdır" yazısına ulaşamaz.

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkış nokta: Bus error ne zaman olur?

Çözüm: Özellikle hizalama isteyen makinelerde int\* adresi 4'ün katı değilse ve int okunursa bus error olabilir. Her mimaride aynı görülmeyebilir.

## 10. Struct, Union ve Padding

Slayt kaynağı: Hafta4: struct yerleşimi ve padding; Hafta7: union aynı anda tek alanı kullanır, boyutu en büyük alan/hizalamaya göre belirlenir.

### Slayttan Ders Notu

| Yapı    | Bellek mantığı                                                                                              |
|---------|-------------------------------------------------------------------------------------------------------------|
| struct  | Alanlar sırayla yerleşir; hizalama için araya padding girebilir. Boyut alanların düz toplamı olmayabilir.   |
| union   | Tüm alanlar aynı bellek başlangıcını paylaşır. Boyut en büyük alan + hizalama kadardır.                     |
| Padding | Sonraki alanın uygun hizaya gelmesi veya struct dizilerinde her elemanın hizalı başlaması için ek boş byte. |

```
struct Test { char c1; int i1; char c2; };
// Genelde: c1=1, padding=3, int=4, c2=1, tail padding=3 => 12 byte

union Test { char c1; int i1; double d1; };
// En büyük alan double ise genelde 8 byte
```

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkış nokta: struct { char c1; int i1; char c2; } ve union { char c1; int i1; double d1; } kaç byte?

Çözüm: Struct genelde 12 byte, union genelde 8 byte. Fotoğraftaki 2023-2024 benzeri soruda bu mantık işaretlenmişti.

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkış nokta: struct { char c1; char c2; int i1; } kaç byte?

Çözüm: char+char=2, int 4 byte hizası için 2 byte padding, int=4; toplam 8 byte. Şıklarda 8 varsa cevap budur.

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkış nokta: union neden struct gibi tüm alanların toplamı değildir?

Çözüm: Union alanları aynı bellek bölgesini paylaşır; aynı anda sadece bir alan anlamlı kabul edilir. Bu yüzden toplam değil en büyük alan belirleyicidir.

## 11. Process, fork, Signal ve Kimlik Fonksiyonları

Slayt kaynağı: Hafta8: sinyal, signal handler, Ctrl-Z/SIGSTOP, Ctrl-C/SIGINT; Hafta8 temel proses ve kimlik fonksiyonları.

## Slayttan Ders Notu

| Fonksiyon / Tuş       | Anlam                                                                |
|-----------------------|----------------------------------------------------------------------|
| getpid()              | Çalışan prosesin process ID değerini verir.                          |
| getppid()             | Parent process ID değerini verir.                                    |
| getuid()              | Kullanıcı ID değerini verir.                                         |
| getgid()              | Kullanıcının grup ID değerini verir.                                 |
| fork()                | Yeni child process oluşturur. Parent'ta child pid, child'da 0 döner. |
| Ctrl+C                | SIGINT gönderir; genelde programı keser.                             |
| Ctrl+Z                | SIGSTOP/SIGTSTP ile askıya alır; fg ile geri alınabilir.             |
| Ctrl+D                | EOF gönderir.                                                        |
| signal(SIGINT, islem) | Ctrl+C gelince islem fonksiyonunu handler olarak çalıştırır.         |

## ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: printf("%d %d %d", getpid(), getuid(), getgid()) hangi sırayı basar?  
 Çözüm: Sırasıyla proses id, kullanıcı id, kullanıcı grup id basılır.

## ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: Parent proses kimliğini hangi fonksiyon verir?  
 Çözüm: getppid() parent process ID değerini verir. getpid() mevcut prosesin kimliğidir.

## ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: Ctrl+C basılınca islem() çalışsın istenirse başlangıçta hangi satır yazılır?  
 Çözüm: signal(SIGINT, islem); kullanılır. kill(SIGINT, islem) doğru değildir; kill proseslere sinyal göndermek içindir.

## ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: Bir programı interrupt etmek/sonlandırmak için hangi tuş?  
 Çözüm: Ctrl+C SIGINT gönderir. Ctrl+Z askıya alır, Ctrl+D EOF gönderir.

## 12. Derleme, Bağlama, Header ve Makefile

Slayt kaynağı: Hafta2 ve makefile slaytları: compile/link ayrımı, gcc -c object üretimi, Makefile kural yapısı.

## Slayttan Ders Notu

| Kavram              | Anlam                                                                    |
|---------------------|--------------------------------------------------------------------------|
| Preprocess          | #include ve #define gibi işlemler.                                       |
| Compile             | .c dosyasını assembly/object seviyesine çevirme.                         |
| Object file         | gcc -c main.c sonucu main.o oluşur.                                      |
| Link                | Object dosyaları ve kütüphaneleri birleştirip executable üretir.         |
| Undefined reference | Genelde link aşamasında çıkar; bildirim var ama tanım bulunamamıştır.    |
| .h dosyası          | Fonksiyon prototipi, struct tanımı, extern bildirimleri için kullanılır. |

| Kavram        | Anlam                                                                 |
|---------------|-----------------------------------------------------------------------|
| extern        | Değişken/fonksiyon başka yerde tanımlı, burada bildiriliyor demektir. |
| static global | Sadece bulunduğu .c dosyasında görünürlük sağlar.                     |

```

hedef : bağımlılıklar
<TAB>komut

proje : main.o io.o
       gcc -o proje main.o io.o

```

#### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: gcc -c main.c ne üretir?

Çözüm: main.o object dosyası üretir; executable üretmek için link aşaması gerekir.

#### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: Undefined reference hangi aşamada çıkar?

Çözüm: Link aşamasında. Derleyici prototipi görmüş olabilir ama gerçek fonksiyon/variable tanımı linker tarafından bulunamamıştır.

#### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: Makefile'da komut satırı nasıl başlamalıdır?

Çözüm: TAB karakteri ile başlamalıdır. Boşluk kullanmak sık yapılan hatadır.

## 13. Pseudo Assembler Temelleri: Register, Komut, Stack

*Slayt kaynağı: Hafta13: r0-r4, sp, fp, pc registerları; Hafta14: jsr/ret ve stack frame; Hafta15: pointer/dizi assembler erişimleri.*

### Slayttan Ders Notu

| Register / Komut | Görevi                                                                         |
|------------------|--------------------------------------------------------------------------------|
| r0-r4            | Genel amaçlı registerlar. Dönüş değeri çoğunlukla r0'dadır.                    |
| sp               | Stack pointer. Yığındaki kullanılabilir/üst bölgeyi takip eder.                |
| fp               | Frame pointer. Aktif fonksiyonun local ve parametrelerine referans noktasıdır. |
| pc               | Program counter. Bir sonraki çalıştırılacak komutun adresini tutar.            |
| jsr a            | Alt programa/fonksiyona dallanır; pc ve fp stack'e push edilir.                |
| ret              | Fonksiyondan döner; fp ve pc stack'ten geri alınır.                            |
| push #4          | Stack'te 4 byte yer açar.                                                      |
| pop #4           | Stack'ten 4 byte geri alır.                                                    |
| ld [adres] -> r0 | Bellekten r0'a yükler.                                                         |
| st r0 -> [adres] | r0 değerini belleğe yazar.                                                     |

#### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkmış nokta: Fonksiyonun dönüş değeri hangi registerda saklanır?

Çözüm: r0. Slaytlardaki örneklerde a() dönüş değeri r0'a konur ve çağırılan fonksiyon r0'dan alır.

#### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkış nokta: Bellekteki kullanılabilir adrese işaret eden register hangisi?

Çözüm: sp yani stack pointer. Fotoğraftaki 2023-2024 sorusunda bu nokta sorulmuştu.

#### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkış nokta: pc neyi tutar?

Çözüm: Program counter; bir sonraki çalıştırılacak komutun adresini tutar.

#### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkış nokta: fp ne işe yarar?

Çözüm: Aktif fonksiyonun stack frame referansıdır. Local değişkenler genellikle fp, fp-4, fp-8; parametreler fp+12, fp+16 gibi adreslerden okunur.

## 14. Pseudo Assembler Stack Offset Kuralları

*Slayt kaynağı: Hafta14 slaytlarında local değişkenler fp çevresinde, parametreler fp+12 ve devamında gösterilir. Hafta15 dizi/pointer örneklerinde p[3] için [fp+12]+12 hesabı yapılır.*

### Slayttan Ders Notu

#### Bu dersteki en önemli assembler kuralı

Tek local değişken varsa çoğu örnekte [fp] kullanılır.

Birden fazla local değişken varsa slayt örneklerinde son local [fp], önceki local [fp-4], daha önceki [fp-8] şeklinde görülür.

Fonksiyon parametreleri genelde [fp+12], [fp+16], [fp+20] şeklindedir.

int 4 byte kabul edilir.

| C tarafı                     | Assembler adres mantığı                    |
|------------------------------|--------------------------------------------|
| int foo(int a, int b)        | a = [fp+12], b = [fp+16]                   |
| int foo(int a, int b, int c) | a = [fp+12], b = [fp+16], c = [fp+20]      |
| foo içinde tek local int d;  | d genellikle [fp]                          |
| main içinde int i,j,k;       | Slayt örneğinde i=[fp-8], j=[fp-4], k=[fp] |
| return değer                 | r0                                         |

#### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkış nokta: int fonk(int a, int b){ int c; c=a+b; return c; } için c local değişken adresi?

Çözüm: Tek local c için ders pseudo-assembler kalıbında c = [fp] kabul edilir. Bu yüzden şıklarda fp varsa cevap fp olur.

#### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkış nokta: Aynı fonksiyonda a ve b parametrelerinin adresleri?

Çözüm: a = fp+12, b = fp+16. Çünkü parametreler caller tarafından stack'e konur ve callee frame'inde pozitif offsettedir.

#### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkış nokta: int foo(int a,int b,int c) fonksiyonunda b parametresi adresi?

Çözüm: a fp+12 ise b fp+16, c fp+20 olur. Fotoğraflardaki b local/parametre soruları bu kalıpla çözülür.

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkış nokta: Global int p; için foo çalışırken p değişkeninin adresi fp ile ifade edilir mi?

Çözüm: Hayır. Global değişken stack frame içinde değildir. Bu yüzden fp, fp-4, fp+4 gibi seçenekler varsa cevap "hiçbiri" olabilir.

## 15. C Kodunu Pseudo Assembler ile Eşleştirme Kalıpları

*Slayt kaynağı: Hafta14-15: fonksiyon çağırısı, parametre push, jsr, pop, r0 dönüş değeri, pointer/dizi referans kaldırma.*

### En çok çıkan kalıplar

```
C: i = 5;
ASM:
mov #5 -> r0
st r0 -> [fp]      # i [fp] ise
```

```
C: x = a(8);
ASM genel mantık:
push #4 veya push #yerel_alan
push #8           # parametreyi stacke koy
jsr a
pop #4            # parametreyi geri al
st r0 -> [fp-4]   # dönüş değerini x değişkenine yaz
```

```
C: a(int *p) { int i; i = p[3]; }
ASM genel mantık:
push #4           # i için yer
ld [fp+12] -> r0   # p değerini yükle
mov #12 -> r1      # 3 * sizeof(int)
add r0, r1 -> r0   # p + 12
ld [r0] -> r0      # *(p+3) yani p[3]
st r0 -> [fp]      # i = p[3]
ret
```

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkış nokta: a(int \*p){ int i; i=p[3]; } için doğru assembler hangisi?

Çözüm: Doğru seçenekte önce p = [fp+12] yüklenir, sonra 12 eklenir, sonra [r0] ile referans kaldırılır, en son r0 [fp] local i'ye yazılır. Sadece mov #3 ekleyen seçenek yanlıştır; int dizide indeks 3, 12 byte demektir.

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkış nokta: main(){ int i,k; i=5; i=a(&i); } için doğru assembler nasıl anlaşılır?

Çözüm: Önce local alan ayrılır, i'ye 5 yazılır, &i adresi stack'e parametre olarak push edilir, jsr a çağırılır, parametre pop edilir, r0 dönüşü tekrar i'nin adresine yazılır.

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkış nokta: if/else assembler eşleştirme sorularında nasıl çözülür?

Çözüm: cmp sonrası bge/blt gibi dallanma hangi bloğa atlıyor takip edilir. C'de if(i<j) x=i; else x=j; ise karşılaştırmadan sonra doğru bloklarda i veya j'nin x'e yazıldığını kontrol et.

## 16. errno, perror ve Hata Mesajı Eşleştirme

*Slayt kaynağı: open/write/stat gibi system call örneklerinde başarısız dönüş -1 ve perror/errno kullanımı.*

| Durum                        | Muhtemel errno/mesaj                          | Çıkış bağlantısı                  |
|------------------------------|-----------------------------------------------|-----------------------------------|
| Açılmamış fd'ye write(3,...) | EBADF - Bad file descriptor / Bad file number | write(3,s,strlen(s)) sorusu       |
| Dosya yokken open O_RDONLY   | ENOENT - No such file or directory            | olmayan dosya soruları            |
| İzin yokken open/read        | EACCES - Permission denied                    | chmod ----- sonrası cat/open      |
| Yetki gerektiren işlem       | EPERM - Operation not permitted               | seçeneklerde gelir                |
| Proses yok                   | ESRCH - No such process                       | kill/process seçeneklerinde gelir |

### ÇIKMIŞ SORU BAĞLANTISI VE ÇÖZÜM

Sorulan çıkış nokta: write: Permission denied mi, Bad file number mı?

Çözüm: fd geçersizse permission değil EBADF/Bad file descriptor olur. Permission denied, fd açılırken izin reddedilirse beklenir.

## 17. Eski Çıkışlardan Gelen Genel Linux/OS Soruları

*Slayt kaynağı: 2009-2010 görselleri ve eski final sayfalarında shell, /etc, GPL/GNU, Ubuntu/Linux gibi genel sorular da görülüyor.*

| Soru tipi                    | Cevap / çözüm mantığı                                                                                 |
|------------------------------|-------------------------------------------------------------------------------------------------------|
| Hangi shell değildir?        | The C shell, Bourne shell vb. gerçektir; "The Gnu shell" gibi uydurma/yanlış seçenek gelebilir.       |
| /etc içinde ne var?          | Sistem konfigürasyon dosyaları.                                                                       |
| ls -a ne listeler?           | Gizli dosyalar dahil tüm dosyalar; . ve .. görünür.                                                   |
| Özgür yazılımın özgürlükleri | Kullanma, inceleme/değiştirme, kopya dağıtma, değiştirilmiş halini dağıtma. Satılamaz demek değildir. |
| Ubuntu nedir?                | Linux dağıtımdır.                                                                                     |
| Linux dağıtımı değildir?     | Unix veya GNU tek başına dağıtım değildir; Ubuntu/Debian dağıtımdır.                                  |
| GNU projesi başlangıcı       | 1983 Richard Stallman bağlamı bilinir.                                                                |

### Not

Bu bölüm güncel sistem programlama kod soruları kadar ağırlıklı görünmüyor; ama eski çıkışlarda kısa kavram sorusu olarak gelmiş.

## 18. Soru Çözme Reçeteleri

### Komut çıktısı sorusu çözme

| Adım | Ne yapacaksın?                          |
|------|-----------------------------------------|
| 1    | Her komutu sırayla çalışmış gibi düşün. |
| 2    | Dosya içerik tablosu tut: f1, f2, f3... |

| Adım | Ne yapacaksın?                                                        |
|------|-----------------------------------------------------------------------|
| 3    | > varsa dosya sıfırlanır, >> varsa eklenir.                           |
| 4    | Hata stdout mu stderr mi ayır.                                        |
| 5    | Hard link varsa iki isim aynı inode olduğu için içerik ortak değişir. |

### Permission sorusu çözme

- 1) Dosya mı izin mi?  
Dosya default 0666, izin default 0777
- 2) umask varsa: gerçek = mode & ~umask
- 3) chmod doğrudan verilen mode'u uygular.
- 4) rwx tablosuna çevir.

### Assembler sorusu çözme

- 1) r0 dönüş değeridir.
- 2) sp stack pointer, fp frame pointer, pc sıradaki komut.
- 3) Parametreler: fp+12, fp+16, fp+20.
- 4) Local değişkenler: fp, fp-4, fp-8.
- 5) Pointer dizide p[3] için 3\*4=12 byte ekle.
- 6) jsr sonrası dönüş r0'dan alınır.

## 19. Cevaplı Mini Test - Çıkmış Tarzı

| Soru                                        | Cevap                                                   |
|---------------------------------------------|---------------------------------------------------------|
| ln -s d1 s1 ne yapar?                       | d1 hedefini gösteren s1 isimli symbolic link oluşturur. |
| ln d1 d2 ne yapar?                          | d1 ile aynı inode'u gösteren hard link d2 oluşturur.    |
| umask 022 ve mode 0666 sonucu?              | 0644 = rw-r--r--.                                       |
| umask 077 ve dosya default sonucu?          | 0600 = rw-----.                                         |
| chmod 600 dosya sonucu?                     | Sadece owner read/write: rw-----.                       |
| open başarısızsa ne döner?                  | -1 döner ve errno set edilir.                           |
| Program başında ilk yeni fd genelde kaçtır? | 3; çünkü 0,1,2 stdin/stdout/stderr açıktır.             |
| write(3,...) ve 3 açık değilse?             | Bad file descriptor / Bad file number.                  |
| fprintf sistem çağırısı mı?                 | Hayır, C library fonksiyonudur.                         |
| stat ne verir?                              | Dosya metadata/inode bilgisi.                           |
| st_size ne?                                 | Dosya boyutu.                                           |
| st_nlink ne?                                | Hard link sayısı.                                       |
| readdir ne döndürür?                        | struct dirent*.                                         |
| de->d_name ne?                              | Dizin girişinin dosya/dizin adı.                        |
| Recursive izin gezerken ne atlanır?         | . ve ...                                                |
| strdup eşdeğeri?                            | malloc(strlen(s)+1) + strcpy.                           |
| strlen("abc\0def")?                         | 3.                                                      |
| NULL pointer dereference sonucu?            | Segmentation fault.                                     |



| Soru                                         | Cevap                                 |
|----------------------------------------------|---------------------------------------|
| struct {char; int; char;} genelde kaç byte?  | 12.                                   |
| union {char; int; double;} genelde kaç byte? | 8.                                    |
| Ctrl+C hangi sinyal?                         | SIGINT.                               |
| signal(SIGINT,islem) ne yapar?               | Ctrl+C gelince islem handler çalışır. |
| getppid ne verir?                            | Parent process ID.                    |
| r0 görevi?                                   | Fonksiyon dönüş değeri.               |
| sp görevi?                                   | Stack pointer.                        |
| fp+12 genelde ne?                            | İlk parametre.                        |
| p[3] assemblerda offset?                     | 3 * sizeof(int) = 12 byte.            |

## 20. Sınava Girmeden Önce Son 10 Satır

- 1) > stdout'u yönlendirir, stderr ekranda kalır; 2> stderr içindir.
- 2) open başarılıysa fd döner, başarısızsa -1.
- 3) İlk yeni fd genelde 3'tür.
- 4) O\_CREAT yoksa dosya yokken oluşturulmaz; O\_TRUNC varsa dosya sıfırlanır.
- 5) gerçek izin = mode & ~umask.
- 6) Hard link aynı inode; soft link hedef yoludur.
- 7) stat metadata verir; readdir isim verir; d\_name dosya adıdır.
- 8) read() string sonuna \0 koymaz.
- 9) Struct padding yüzünden büyür; union en büyük alan kadardır.
- 10) r0 dönüş, sp stack, fp frame, pc sıradaki komut; parametreler fp+12'den başlar.

### Son tavsiye

Çıkış soruların çoğu aynı mantığın farklı giydirilmiş hali: dosyanın içeriğini, permission hesabını, fd durumunu veya fp/sp/r0 adresini adım adım takip et.